Ansible ⭐ ⭐ — 30 Scenario-Based Q&A

1. One server fails during Ansible playbook execution. What will you do?
First, I check the Ansible output and identify which task failed on that server.
Then I compare the failed server with a working server and check connectivity, configuration, permissions, packages, and variables.
I identify the actual difference, fix the issue, and run the playbook again.
Finally, I verify that the task completed successfully.
Simple ga gurthupettuko: Oka server lo problem vaste, first ye task fail ayyindo chusi, working server tho compare chesi, problem enti ani find chesi, fix chesi playbook malli run cheyyali.
Eng: If one server fails, first check which task failed, compare it with a working server, find the problem, fix it, and run the playbook again.

---

2. Ansible cannot connect to a server. What will you check?
First, I check whether the server is reachable from the Ansible control machine.
Then I verify the hostname or IP, SSH connectivity, port 22, username, SSH key, and firewall rules.
I also check whether the server is running and whether the correct inventory entry is being used.
After fixing the connectivity issue, I test the connection again.
Simple ga gurthupettuko: Server ki Ansible connect avvakapothe, first server reachable ga undho chusi, SSH, IP, username, key, port, firewall anni check chesi connection fix cheyyali.
Eng: If Ansible cannot connect to a server, check reachability, SSH, IP, username, key, port, and firewall, then test the connection again.

---

3. Ansible playbook suddenly started failing. What will you check?
First, I check the exact task and error message from the latest execution.
Then I compare the current run with the last successful run and check recent changes in the playbook, inventory, variables, server configuration, packages, or credentials.
I identify which change caused the failure and fix it.
After that, I rerun the playbook and verify the result.
Simple ga gurthupettuko: Playbook sudden ga fail ayithe, first error unna task chusi, last successful run tho compare chesi, recent change enti ani find chesi fix cheyyali.
Eng: If the playbook suddenly fails, check the failed task, compare it with the last successful run, find the recent change, fix it, and rerun the playbook.

---

4. Ansible says "Permission denied". What will you check?
First, I check which operation is getting the permission error.
Then I verify the remote user, file or directory permissions, ownership, and whether the task requires elevated privileges.
If required, I use become with the correct sudo configuration.
I then rerun the task and verify the permissions.
Simple ga gurthupettuko: Permission denied vaste, ye file or command ki permission kavalo chusi, user, ownership, permissions, sudo requirement check chesi fix cheyyali.
Eng: If Ansible shows permission denied, check the user, ownership, permissions, and sudo requirement, then fix the permission and rerun the task.

---

5. Ansible become **is failing. What will you check?**
First, I check whether the remote user is allowed to use sudo.
Then I verify the sudo configuration, become settings, password requirements, and the user specified in the inventory.
I also check whether the command actually requires elevated privileges.
After correcting the configuration, I run the task again.
Simple ga gurthupettuko: Become fail ayithe, remote user ki sudo access undho chusi, sudo configuration and become settings correct ga unnayo check cheyyali.
Eng: If become fails, check whether the remote user has sudo access and verify the sudo and become configuration before running the task again.

---

6. Ansible task works on one server but fails on another. What will you do?
First, I compare the two servers because the playbook is the same.
I check OS version, installed packages, permissions, configuration files, variables, disk space, and network connectivity.
I identify the server-specific difference causing the problem.
Then I fix that difference and rerun the task.
Simple ga gurthupettuko: Oka server lo work ayyi inkoka server lo fail ayithe, rendu servers ni compare chesi difference enti ani find chesi aa problem fix cheyyali.
Eng: If a task works on one server but fails on another, compare both servers, find the difference, fix it, and rerun the task.

---

7. Ansible playbook is taking too long. What will you check?
First, I identify which task or group of servers is taking most of the time.
Then I check SSH connection time, package installation, file transfers, remote commands, and network delays.
I also check whether unnecessary tasks are running sequentially.
If possible, I optimize the tasks and use suitable parallel execution.
Simple ga gurthupettuko: Playbook slow ayithe, first ye task ekkuva time tiskuntundo chusi, network and remote operations check chesi unnecessary work reduce cheyyali.
Eng: If the playbook is slow, find the slow task, check network and remote operations, remove unnecessary work, and optimize execution.

8. Ansible playbook is running sequentially and taking too much time. How can you improve it?
I first identify tasks that do not depend on each other.
Those tasks can be executed in parallel where appropriate.
I also reduce unnecessary remote calls and avoid repeating the same operation.
After optimization, I compare the new execution time with the previous run.
Simple ga gurthupettuko: Independent tasks unte vatini parallel ga run chesi, repeated work remove chesi playbook execution time tagginchali.
Eng: If tasks are independent, run them in parallel where appropriate, remove repeated work, and reduce the overall playbook execution time.

---

9. Ansible reports "No hosts matched". What will you check?
First, I check the inventory file and verify whether the required server exists in it.
Then I check the group name used in the playbook against the actual inventory group name.
I also verify the inventory file path being used during execution.
After correcting the inventory or group name, I run the playbook again.
Simple ga gurthupettuko: No hosts matched vaste, inventory lo server undho, group name correct ga undho, correct inventory file use chestunnamo check cheyyali.
Eng: If Ansible says no hosts matched, check the inventory, group name, and inventory file path, then correct the mismatch.

---

10. Ansible is using the wrong server. What will you check?
First, I check the inventory file and the host or group specified in the playbook.
Then I verify host variables, aliases, and inventory group assignments.
I also check whether the command is using the expected inventory file.
After correcting the inventory mapping, I run the playbook again.
Simple ga gurthupettuko: Wrong server meeda run ayithe, playbook group, inventory, host names, and inventory file correct ga unnayo check cheyyali.
Eng: If Ansible runs against the wrong server, check the playbook group, inventory hosts, aliases, and inventory file being used.

---

11. Ansible variable has the wrong value. What will you check?
First, I identify where the variable is defined.
Then I check group variables, host variables, playbook variables, role variables, extra variables, and default values.
I determine which value has higher precedence and is overriding the expected value.
Then I correct the variable source and rerun the playbook.
Simple ga gurthupettuko: Variable wrong ga vaste, aa variable ekkada define ayyindo chusi, vere value danini override chestundo find chesi correct cheyyali.
Eng: If a variable has the wrong value, find where it is defined, identify which value is overriding it, correct it, and rerun the playbook.

---

12. Same variable is defined in multiple places. Which one will Ansible use?
I first identify all locations where the variable is defined.
Ansible applies variable precedence rules, so some definitions have higher priority than others.
I check whether an extra variable, host variable, group variable, role variable, or default value is overriding the expected value.
Then I remove unnecessary duplicate definitions.
Simple ga gurthupettuko: Same variable chala places lo unte, Ansible priority rules prakaram higher priority value use chestundi, kabatti override ekkada jarugutundo find cheyyali.
Eng: When a variable exists in multiple places, Ansible uses its precedence rules, so I find which higher-priority value is overriding the expected one.

---

13. Ansible task shows "changed" every time. What is the problem?
This usually means the task is not behaving idempotently.
I check whether the command runs every time without checking the current state.
I prefer Ansible modules with proper state management instead of repeatedly running shell commands.
If necessary, I use conditions or appropriate change detection.
Simple ga gurthupettuko: Every run lo task changed ani vaste, current state check cheyyakunda command repeat avutundo chusi, idempotent ga change cheyyali.
Eng: If a task shows changed every time, check whether it is repeating an operation unnecessarily and make the task idempotent.

---

14. What is idempotency, and why is it important in Ansible?
Idempotency means running the same playbook multiple times should not create unnecessary changes.
For example, if a package is already installed, Ansible should report no change on the next run.
This makes automation predictable and safer.
I prefer Ansible modules that manage the desired state.
Simple ga gurthupettuko: Playbook enni sarlu run chesina already correct ga unna setup ni malli unnecessary ga change cheyyakudadhu; danine idempotency antam.
Eng: Idempotency means repeated playbook runs should not make unnecessary changes when the system is already in the desired state.

---

15. Ansible handler is not running. What will you check?
First, I check whether the task actually changed anything.
Handlers normally run when the related task sends a notification.
I verify the handler name and the notify statement match correctly.
I also check whether the handler is being triggered at the expected point in the playbook.

Simple ga gurthupettuko: Handler run avvakapothe, main task actual ga change ayyinda chusi, notify and handler names correct ga match ayyayo check cheyyali.
Eng: If a handler does not run, check whether the task changed anything and verify that the notify name matches the handler correctly.

---

16. Ansible template is not generating the expected configuration. What will you check?
First, I check the template file and the variable names used inside it.
Then I verify whether the expected variables are available on the target host.
I also check indentation, Jinja2 syntax, conditions, and default values.
After fixing the template or variable issue, I run the playbook again.
Simple ga gurthupettuko: Template wrong output isthe, template variables, Jinja syntax, indentation, and actual variable values check chesi correct cheyyali.
Eng: If a template generates the wrong configuration, check the template syntax, variables, indentation, and values, then fix and rerun it.

---

17. Ansible package installation is failing. What will you check?
First, I check the exact package-manager error from the playbook output.
Then I verify the package name, repository configuration, OS version, network connectivity, and package-manager availability.
I also check whether another package operation is currently running.
After fixing the repository or package issue, I rerun the task.
Simple ga gurthupettuko: Package install fail ayithe, package name, repository, OS, network, and package manager problem undho check chesi fix cheyyali.
Eng: If package installation fails, check the package name, repository, OS, network, and package-manager status before rerunning the task.

---

18. Ansible service failed to start. What will you do?
First, I check the Ansible output and verify the service status on the target server.
Then I check the service logs and configuration file for the actual startup error.
I also verify dependencies, ports, permissions, and required files.
After fixing the root cause, I start the service and verify its health.
Simple ga gurthupettuko: Service start avvakapothe, service status and logs chusi, configuration or dependency problem fix chesi service healthy ga undho verify cheyyali.
Eng: If a service fails to start, check its status and logs, fix the configuration or dependency problem, and verify that the service becomes healthy.

---

19. Ansible Vault secret is not working. What will you check?
First, I verify whether the correct encrypted file is being used.
Then I check the Vault password or password file and whether the playbook is receiving it correctly.
I also verify that the required variable exists inside the encrypted file.
After correcting the Vault configuration, I rerun the playbook.
Simple ga gurthupettuko: Vault secret work avvakapothe, correct encrypted file, Vault password, password file, and secret variable correct ga unnayo check cheyyali.
Eng: If an Ansible Vault secret is not working, verify the encrypted file, Vault password, password file, and required secret variable.

---

20. How do you safely manage production secrets in Ansible?
I avoid storing passwords or tokens directly in normal playbooks or inventory files.
I use Ansible Vault or an approved external secret-management solution.
Access to production secrets is restricted based on least privilege.
I also avoid printing secret values in task output or logs.
Simple ga gurthupettuko: Production secrets ni plain text lo pettakunda Vault or secure secret system use chesi, required users ki matrame access ivvali.
Eng: I keep production secrets out of plain text files, store them securely with Vault or a secret manager, and restrict access.

---

21. Ansible playbook changed configuration but service did not restart. What will you check?
First, I check whether the configuration task actually reported a change.
Then I verify whether the task has the correct notify statement connected to the service restart handler.
I also check the handler name and service configuration.
After fixing the notification or handler, I rerun the playbook.
Simple ga gurthupettuko: Configuration change ayina service restart kakapothe, task change ayyinda, notify correct ga undha, handler run ayyinda ani check cheyyali.
Eng: If the service does not restart after a configuration change, verify the task change, notification, handler, and service configuration.

---

22. Ansible role works in DEV but fails in PROD. What will you check?
I compare the DEV and PROD servers for OS versions, packages, variables, permissions, credentials, network access, and configuration.
I also check whether the same role version is being used in both environments.
I identify the environment-specific difference and fix it.
Then I test the role again in the target environment.
Simple ga gurthupettuko: DEV lo work ayyi PROD lo fail ayithe, rendu environments lo OS, packages, variables, permissions, network, and role version compare cheyyali.

Eng: If an Ansible role works in DEV but fails in PROD, compare both environments, find the difference, fix it, and test again.

---

23. Ansible is failing because a required command is not found. What will you check?

First, I check whether the required package or binary exists on the target server.
Then I verify the executable path and environment variables such as PATH.
I also check whether the playbook installs the required dependency before using it.
After installing or correcting the path, I rerun the task.
Simple ga gurthupettuko: Command not found vaste, required package install ayyinda, binary ekkada undho, PATH correct ga undho check chesi fix cheyyali.
Eng: If a command is not found, verify the required package, binary path, and PATH, then install or correct the dependency.

24. Ansible playbook works manually but fails from Jenkins. Why?

I compare the Jenkins execution environment with the manual environment.
I check the SSH key, inventory path, Ansible version, environment variables, working directory, permissions, and credentials.
Jenkins may also be using a different user or agent.
I identify the environment difference and correct it.
Simple ga gurthupettuko: Manual ga work ayyi Jenkins lo fail ayithe, Jenkins user, agent, credentials, inventory, Ansible version, and environment differences check cheyyali.
Eng: If Ansible works manually but fails in Jenkins, compare the Jenkins user, agent, credentials, inventory, Ansible version, and environment.

25. Ansible playbook partially completed and then failed. What will you do?

First, I identify which hosts and tasks completed successfully.
Then I identify exactly where the failure occurred and whether the completed changes are safe.
I fix the root cause and rerun the playbook because idempotent tasks should safely converge the remaining servers.
I verify the final state across all affected servers.
Simple ga gurthupettuko: Playbook half lo fail ayithe, already em changes ayyayo chusi, failure fix chesi playbook malli run chesi anni servers correct state lo unnayo verify cheyyali.
Eng: If a playbook partially fails, check what already changed, fix the failure, rerun it safely, and verify the final state on all servers.

26. Ansible inventory has hundreds of servers. How will you manage it?

I organize servers into meaningful groups based on environment, application, or role.
I use group variables and host variables for configuration that differs between servers.
For dynamic environments, I can use dynamic inventory or cloud inventory plugins.
This keeps the inventory manageable and reduces duplicate configuration.
Simple ga gurthupettuko: Chala servers unte, environment and application based groups create chesi, common values group variables lo maintain chesi inventory ni organized ga petkovali.
Eng: For many servers, I organize hosts into meaningful groups, use group variables, and use dynamic inventory when appropriate.

27. Ansible playbook is changing too many servers unexpectedly. What will you check?

First, I stop the execution if it is still running and assess the impact.
Then I check the inventory, host pattern, variables, and --limit or targeting options used during execution.
I verify whether the playbook selected a broader group than intended.
After confirming the correct target, I rerun it with controlled scope.
Simple ga gurthupettuko: Unexpected servers change avutunte, execution stop chesi inventory and host pattern check chesi correct servers ki matrame playbook run cheyyali.
Eng: If Ansible targets unexpected servers, stop the execution, verify the inventory and host pattern, and rerun it only against the intended servers.

28. How do you safely test an Ansible playbook before Production?

I first test the playbook in a lower environment that closely matches Production.
I use check mode where appropriate to understand expected changes before applying them.
I review the output and validate the changes on a small set of servers first.
Only after successful validation do I proceed with Production execution.
Simple ga gurthupettuko: Production mundu lower environment lo test chesi, possible ayithe check mode use chesi, small set servers meeda verify chesi tarvatha PROD ki vellali.
Eng: Before Production, test in a lower environment, review the expected changes, validate on a small set of servers, and then proceed safely.

29. How do you troubleshoot an Ansible playbook in Production?

First, I identify the failed task and assess the Production impact.
Then I check the Ansible output, target-server logs, recent changes, inventory, variables, and permissions.
I avoid making unnecessary changes directly in Production and choose the safest recovery approach.
After fixing the root cause, I rerun only the required scope and verify the service.
Simple ga gurthupettuko: PROD lo Ansible fail ayithe, first impact check chesi, failed task and logs chusi, recent changes identify chesi, safe ga fix chesi service verify cheyyali.
Eng: In Production, I first assess the impact, identify the failed task, check logs and recent changes, fix the issue safely, and verify the service.

30. How do you design Production-ready Ansible automation?

I use reusable roles, organized inventories, proper variable management, secure secrets, and idempotent tasks.
I test changes in lower environments before Production.
I also use version control, code review, controlled execution, and clear rollback or recovery procedures.
This makes automation consistent, repeatable, and safer.
Simple ga gurthupettuko: Production Ansible ki reusable roles, clean inventory, secure secrets, idempotent tasks, testing, code review, and safe recovery process undali.
Eng: Production Ansible should use reusable roles, organized inventory, secure secrets, idempotent tasks, testing, code review, and safe recovery procedures.